

MA 5755: Data Analysis & Visualization in R/Python/SQL

Week 12: SQL

Dr. Rakhi Singh
Department of Mathematics
IIT Madras

Spring 2026

What is SQL and Why Use It?

- SQL (Structured Query Language) is used to work with data in databases
- Retrieve, filter, combine, and analyze large datasets

Where is it used?

- Research data (experiments, simulations)
- Large structured datasets
- Combining multiple data sources

Example tasks:

- Find all records where a measurement exceeds a threshold
- Combine data from multiple sources to study patterns
- Extract specific subsets of large datasets efficiently

SQL is a fundamental tool for working with structured data at scale

Learning SQL Through Practice

- SQL is learned by writing and [testing queries](#)
- Practice environments provide instant feedback

Platforms:

- SQLBolt (starting point)
- HackerRank
- Mode Analytics SQL Tutorial
- DB Fiddle

Approach:

- Guided exercises
- Experimentation
- Work with realistic datasets

Why Not Just Use CSV Files?

Why Not Just Use CSV Files?

Suppose you have: `students.csv`, `courses.csv`, `enrollments.csv`

Why Not Just Use CSV Files?

Suppose you have: students.csv, courses.csv, enrollments.csv

- How would you list student names along with their courses?
- What if an enrollment refers to a non-existent student ID?
- How do you search efficiently in millions of rows?
- What if multiple users edit the file at the same time?
- How do you avoid duplicate or inconsistent data?

Why Not Just Use CSV Files?

Suppose you have: students.csv, courses.csv, enrollments.csv

- How would you list student names along with their courses?
- What if an enrollment refers to a non-existent student ID?
- How do you search efficiently in millions of rows?
- What if multiple users edit the file at the same time?
- How do you avoid duplicate or inconsistent data?

Answers:

Why Not Just Use CSV Files?

Suppose you have: students.csv, courses.csv, enrollments.csv

- How would you list student names along with their courses?
- What if an enrollment refers to a non-existent student ID?
- How do you search efficiently in millions of rows?
- What if multiple users edit the file at the same time?
- How do you avoid duplicate or inconsistent data?

Answers:

- No built-in relationships between files (no way to link related data)
- Combining data requires manual matching (error-prone)
- No constraints to enforce correctness (inconsistent data can appear)
- Performance degrades at scale (slow searches and inefficient)

Databases provide: (1) Structured, related tables, (2) Enforced consistency, and (3) Efficient querying using SQL.

What is a Database? Tables and Structure

- A database is an organized collection of related data designed for efficient storage, retrieval, and management.
- Data is stored in **tables**, where each table represents one type of entity (e.g., students, courses).
- Each table consists of **rows** (records) and **columns** (attributes).
- Tables are not isolated; they are **related** through common fields (e.g., student_id).
- These relationships allow data from multiple tables to be combined systematically and reliably.

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92

Keys in a Database

- Keys are attributes used to uniquely identify and relate records.
- Keys ensure data consistency and enable relationships between tables.
- A **Primary Key** uniquely identifies each row in a table. No duplicates and no null values.
- A **Foreign Key** links one table to another.
- A **Composite Key** uses multiple columns together to uniquely identify a row.

Identify a possible key:

- Students (student_id, name, email), Enrollments (student_id, course_id, semester), Transactions (account_id, timestamp, amount)
- Which column(s) can uniquely identify each student in **Students**?
- Is a single column enough, or do we need a combination in **Enrollments**?
- What combination would uniquely identify each transaction in **Transactions**?

Basic SQL Query: Retrieving and Filtering Data

- SQL queries specify **what data to retrieve** and **from where**.
- The simplest query uses **SELECT** and **FROM**.

```
SELECT name FROM students;
```

- **SELECT** specifies the column(s) to retrieve.
- **FROM** specifies the table being queried.
- The result is a new table containing only the requested data.
- SQL allows selecting only those records that satisfy given conditions.
- This is done using the **WHERE** clause.

```
SELECT * FROM students WHERE marks > 80;
```

- The condition restricts the output to matching rows.
- This enables precise extraction of relevant subsets of data.

Conditions

Operator	Condition	Example
=	Case sensitive exact string comparison (notice the single equals)	col_name = "abc"
!= or <>	Case sensitive exact string inequality comparison	col_name != "abcd"
LIKE	Case insensitive exact string comparison	col_name LIKE "ABC"
NOT LIKE	Case insensitive exact string inequality comparison	col_name NOT LIKE "ABCD"
%	Used anywhere in a string to match a sequence of zero or more characters (only with LIKE or NOT LIKE)	col_name LIKE "%AT%" (matches " <u>A</u> T", " <u>A</u> ITIC", "CAT" or even "B <u>A</u> TS")
_	Used anywhere in a string to match a single character (only with LIKE or NOT LIKE)	col_name LIKE "AN_" (matches " <u>A</u> ND", but not "AN")
IN (...)	String exists in a list	col_name IN ("A", "B", "C")
NOT IN (...)	String does not exist in a list	col_name NOT IN ("D", "E", "F")

Practice - 1

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

Exercise 3 — Tasks

1. Find all the Toy Story movies ✓
2. Find all the movies directed by John Lasseter ✓
3. Find all the movies (and director) not directed by John Lasseter ✓
4. Find all the WALL-* movies ✓

Practice - 1

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

Exercise 3 — Tasks

1. Find all the Toy Story movies ✓
2. Find all the movies directed by John Lasseter ✓
3. Find all the movies (and director) not directed by John Lasseter ✓
4. Find all the WALL-* movies ✓

```
SELECT * FROM movies WHERE Title LIKE "%Toy Story%"
```

Practice - 1

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

Exercise 3 — Tasks

1. Find all the Toy Story movies ✓
2. Find all the movies directed by John Lasseter ✓
3. Find all the movies (and director) not directed by John Lasseter ✓
4. Find all the WALL-* movies ✓

```
SELECT * FROM movies WHERE Title LIKE "%Toy Story%"
```

```
SELECT * FROM movies WHERE Director LIKE "John Lasseter"
```

Practice - 1

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

Exercise 3 — Tasks

1. Find all the Toy Story movies ✓
2. Find all the movies directed by John Lasseter ✓
3. Find all the movies (and director) not directed by John Lasseter ✓
4. Find all the WALL-* movies ✓

```
SELECT * FROM movies WHERE Title LIKE "%Toy Story%"
```

```
SELECT * FROM movies WHERE Director LIKE "John Lasseter"
```

```
SELECT * FROM movies WHERE Director NOT LIKE "John Lasseter"
```


Practice - 1

Table: Movies

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116
7	Cars	John Lasseter	2006	117
8	Ratatouille	Brad Bird	2007	115
9	WALL-E	Andrew Stanton	2008	104
10	Up	Pete Docter	2009	101

Exercise 3 — Tasks

1. Find all the Toy Story movies ✓
2. Find all the movies directed by John Lasseter ✓
3. Find all the movies (and director) not directed by John Lasseter ✓
4. Find all the WALL-* movies ✓

```
SELECT * FROM movies WHERE Title LIKE "%Toy Story%"
```

```
SELECT * FROM movies WHERE Director LIKE "John Lasseter"
```

```
SELECT * FROM movies WHERE Director NOT LIKE "John Lasseter"
```

```
SELECT * FROM movies WHERE Title LIKE "WALL-%";
```

Ordering and Limiting Results

ORDER BY sorts the result based on one or more columns. DESC = descending (highest first), ASC = ascending (default)

```
SELECT name, marks FROM students  
ORDER BY marks DESC;
```

LIMIT restricts the number of rows returned.

```
SELECT * FROM students  
LIMIT 5;
```

OFFSET skips a specified number of rows (useful for pagination).

Example: skip first 5 rows, then return next 5 rows:

```
SELECT * FROM students  
ORDER BY marks DESC  
LIMIT 5 OFFSET 5;
```

Unique rows and Combining Data from Multiple Tables

DISTINCT removes duplicate values from the result.

```
SELECT DISTINCT course_id FROM enrollments;
```

Real datasets are often split across multiple related tables. SQL provides **JOIN** operations to combine such data based on common fields.

```
SELECT students.name, courses.title  
FROM students  
JOIN enrollments  
ON students.id = enrollments.student_id;
```

- JOIN connects rows from different tables using matching identifiers.
- This directly addresses the difficulty of combining separate CSV files.

JOINS: Basic Types

- JOIN operations combine rows from multiple tables using a related column.
- **INNER JOIN:** returns only rows with matching values in both tables

```
SELECT * FROM A  
INNER JOIN B ON A.id = B.id;
```

- **LEFT JOIN:** returns all rows from left table + matching rows from right
- Non-matching rows in right table appear as NULL

```
SELECT * FROM A  
LEFT JOIN B ON A.id = B.id;
```

- **Key idea:** focus on which table's rows are fully preserved

JOINS: Additional Types

- **RIGHT JOIN:** returns all rows from right table + matching from left

```
SELECT * FROM A  
RIGHT JOIN B ON A.id = B.id;
```

- **FULL OUTER JOIN:** returns all rows from both tables
- Non-matching rows are filled with NULLs

```
SELECT * FROM A  
FULL OUTER JOIN B ON A.id = B.id;
```

- These JOINS allow flexible combination of related datasets
- Choice of JOIN depends on whether unmatched rows should be included

Practice-2

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
5	8.2	380843261	555900000
14	7.4	268492764	475066843
8	8	206445654	417277164
12	6.4	191452396	368400000

- Find the domestic and international sales for each movie

Practice-2

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
5	8.2	380843261	555900000
14	7.4	268492764	475066843
8	8	206445654	417277164
12	6.4	191452396	368400000

- Find the domestic and international sales for each movie

```
SELECT Movies.Title, Boxoffice.Domestic_sales,  
Boxoffice.international_sales FROM Boxoffice  
JOIN Movies ON Movies.Id = Boxoffice.Movie_ID;
```

- Show the sales numbers for each movie that did better internationally rather than domestically

Practice-2

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
5	8.2	380843261	555900000
14	7.4	268492764	475066843
8	8	206445654	417277164
12	6.4	191452396	368400000

Practice-2

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
5	8.2	380843261	555900000
14	7.4	268492764	475066843
8	8	206445654	417277164
12	6.4	191452396	368400000

```
SELECT Movies.Title, Boxoffice.Domestic_sales,  
Boxoffice.international_sales FROM Boxoffice JOIN  
Movies ON Movies.Id = Boxoffice.Movie_ID WHERE  
Boxoffice.international_sales > Boxoffice.Domestic_sales;
```

- List all the movies by their ratings in descending order

Practice-2

Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
5	8.2	380843261	555900000
14	7.4	268492764	475066843
8	8	206445654	417277164
12	6.4	191452396	368400000

```
SELECT Movies.Title, Boxoffice.Domestic_sales,  
Boxoffice.international_sales FROM Boxoffice JOIN  
Movies ON Movies.Id = Boxoffice.Movie_ID WHERE  
Boxoffice.international_sales > Boxoffice.Domestic_sales;
```

- List all the movies by their ratings in descending order

```
SELECT Movies.Title, Boxoffice.Rating FROM Boxoffice  
JOIN Movies ON Movies.Id = Boxoffice.Movie_ID ORDER  
BY Rating DESC
```

Using Expressions in SQL

- SQL allows using **expressions** to compute or transform values during a query.
- Expressions can include arithmetic operations, functions, and combinations of columns.

Example:

```
SELECT particle_speed / 2.0 AS half_particle_speed  
FROM physics_data  
WHERE ABS(particle_position) * 10.0 > 500;
```

- Expressions can be used in both SELECT and WHERE clauses.
- They allow filtering and transforming data without extra processing.
- **Good practice:** Use **AS** to assign clear, descriptive names to computed columns.

Aggregate Functions in SQL

- Aggregate functions compute a **single value** from multiple rows.
- Useful for summarizing datasets (counts, averages, totals).

Common aggregate functions:

- COUNT() — number of rows
- SUM() — total of values
- AVG() — average value
- MIN(), MAX() — smallest / largest values

Example:

```
SELECT COUNT(*) FROM movies;
```

- Aggregates operate over all selected rows and return a single result

Aggregates with Conditions

- Aggregates can be combined with **WHERE** to restrict input data.
- Filtering happens **before** aggregation.

Example:

```
SELECT AVG(rating)
FROM movies
WHERE rating > 7;
```

- **GROUP BY** divides rows into groups based on column values.
- Aggregate functions are then applied **within each group**.

Example:

```
SELECT role, AVG(years_employed)
FROM employees
GROUP BY role;
```

GROUP BY and Filtering Groups: HAVING Clause

- **GROUP BY** Produces one result per group (e.g., average per role)
- Converts “single summary” to “summary per category”
- **WHERE** filters rows **before grouping**
- **HAVING** filters groups **after aggregation**

Example:

```
SELECT role, COUNT(*)  
FROM employees  
GROUP BY role  
HAVING COUNT(*) > 5;
```

- HAVING allows conditions on aggregated values
- Necessary because grouping happens before this step

Key Idea: Aggregation Workflow

- Typical order of operations:
- FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT

Interpretation:

- First select relevant rows (WHERE)
- Then group them (GROUP BY)
- Then compute aggregates
- Finally filter groups (HAVING)

Core idea:

- Aggregates turn raw data into summaries
- GROUP BY + HAVING enables structured analysis over categories

Practice-3

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8

1. Find the longest time that an employee has been at the studio

Practice-3

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8

1. Find the longest time that an employee has been at the studio

```
SELECT MAX(Years_employed) FROM employees;
```

2. For each role, find the average number of years employed by employees in that role

Practice-3

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8

1. Find the longest time that an employee has been at the studio

```
SELECT MAX(Years_employed) FROM employees;
```

2. For each role, find the average number of years employed by employees in that role

```
SELECT Role, AVG(Years_employed) FROM employees GROUP BY Role;
```

3. Find the total number of employee years worked in each building

Practice-3

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8

1. Find the longest time that an employee has been at the studio

```
SELECT MAX(Years_employed) FROM employees;
```

2. For each role, find the average number of years employed by employees in that role

```
SELECT Role, AVG(Years_employed) FROM employees GROUP BY Role;
```

3. Find the total number of employee years worked in each building

```
SELECT Building, Total(Years_employed) FROM  
employees GROUP BY Building;
```

Practice-3

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8

4. Find the number of Artists in the studio (without a HAVING clause)

Practice-3

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8

4. Find the number of Artists in the studio (without a HAVING clause)

```
SELECT COUNT(*) FROM employees WHERE  
Role = "Artist";
```

5. Find the number of Employees of each role in the studio

Practice-3

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8

4. Find the number of Artists in the studio (without a HAVING clause)

```
SELECT COUNT(*) FROM employees WHERE  
Role = "Artist";
```

5. Find the number of Employees of each role in the studio

```
SELECT Role, COUNT(*) FROM employees GROUP BY Role;
```

6. Find the total number of years employed by all Engineers

Practice-3

Table: Employees

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2
Artist	Sherman D.	2w	8

4. Find the number of Artists in the studio (without a HAVING clause)

```
SELECT COUNT(*) FROM employees WHERE  
Role = "Artist";
```

5. Find the number of Employees of each role in the studio

```
SELECT Role, COUNT(*) FROM employees GROUP BY Role;
```

6. Find the total number of years employed by all Engineers

```
SELECT SUM(Years_employed) FROM employees WHERE  
Role = "Engineer"
```

Complete Query

```
SELECT DISTINCT column, AGG_FUNC(column_or_expression), . . .  
FROM mytable  
JOIN another_table  
ON mytable.column = another_table.column  
WHERE constraint_expression  
GROUP BY column  
HAVING constraint_expression  
ORDER BY column ASC/DESC  
LIMIT count OFFSET COUNT;
```

- Not every SQL query requires all available clauses
- SQL is flexible: queries can be as simple or as complex as needed
- Data can be manipulated directly within queries
- Reduces the need for additional programming or post-processing

Inserting Data: INSERT INTO

- SQL uses the **INSERT INTO** statement to add new rows to a table.
- You must specify: table to insert into, and values for each column.
- Basic syntax (all columns):

```
INSERT INTO mytable  
VALUES (value1, value2, ...);
```

- Each row should provide values for all columns (in correct order)
- Multiple rows can be inserted in a single query

```
INSERT INTO mytable  
VALUES (v1, v2), (v3, v4);
```

Inserting with Specific Columns

- You can insert into only selected columns (useful for partial data).
- Columns must be explicitly listed.

```
INSERT INTO mytable (col1, col2)  
VALUES (value1, value2);
```

- Number of values must match number of columns specified
- Remaining columns take default values (if defined)
- **Why this form is useful:**
 - More robust to schema changes
 - Safer when tables have many columns
- **Key idea:**
 - INSERT adds new rows (does not modify existing data)

Updating Existing Data

- The **UPDATE** statement is used to modify existing rows in a table.
- You must specify:
 - The table to update
 - The column(s) and new value(s)
 - Which rows to update (using WHERE)

```
UPDATE mytable  
SET column = value, other_column = value  
WHERE condition;
```

- Updates are applied to all rows satisfying the condition
- If **WHERE is omitted**, all rows are updated (dangerous)

Good practice:

- First test the condition using SELECT before applying UPDATE

Deleting Data

- The **DELETE** statement removes rows from a table.
- You must specify which rows to delete using a condition.

```
DELETE FROM mytable  
WHERE condition;
```

- Only rows satisfying the condition are removed
- If **WHERE is omitted**, all rows are deleted

Example:

```
DELETE FROM movies  
WHERE year < 2005;
```

- DELETE permanently removes data (unlike UPDATE, which modifies it)

Dropping Tables

- The **DROP TABLE** statement removes an entire table from the database.
- This includes:
 - All data in the table
 - The table structure (schema)
- Unlike DELETE, this removes the table itself, not just its rows.

`DROP TABLE mytable;`

- The operation is **permanent and irreversible**
- Often used when a table is no longer needed or during cleanup

Safer variant:

`DROP TABLE IF EXISTS mytable;`

- Avoids errors if the table does not exist
- DELETE removes rows; DROP TABLE removes the entire table

Altering Tables

- SQL allows modifying an existing table structure
- Used when data requirements change over time (new columns, schema updates)

Common operations:

- Add a new column (with type, constraints, default)
- Remove an existing column (not supported in all systems)
- Rename the table

```
ALTER TABLE mytable  
ADD column DataType DEFAULT value;
```

```
ALTER TABLE mytable  
DROP column_name;
```

```
ALTER TABLE mytable  
RENAME TO new_table_name;
```

Creating Tables

- New tables are created using the **CREATE TABLE** statement.
- A table is defined by its **schema** (columns, types, constraints).

```
CREATE TABLE mytable (  
    column DataType Constraint DEFAULT value,  
    another_column DataType ...  
);
```

- Each column specifies:
 - Name
 - Data type (e.g., INT, TEXT, FLOAT)
 - Optional constraints (e.g., NOT NULL, PRIMARY KEY) and default values
- Defines how data will be stored and validated

Creating Tables: Practical Notes

- Use **IF NOT EXISTS** to avoid errors if the table already exists

```
CREATE TABLE IF NOT EXISTS mytable (...);
```

Data type	Description
INTEGER , BOOLEAN	The integer datatypes can store whole integer values like the count of a number or an age. In some implementations, the boolean value is just represented as an integer value of just 0 or 1.
FLOAT , DOUBLE , REAL	The floating point datatypes can store more precise numerical data like measurements or fractional values. Different types can be used depending on the floating point precision required for that value.
CHARACTER(num_chars) , VARCHAR(num_chars) , TEXT	The text based datatypes can store strings and text in all sorts of locales. The distinction between the various types generally amount to underlying efficiency of the database when working with these columns. Both the CHARACTER and VARCHAR (variable character) types are specified with the max number of characters that they can store (longer values may be truncated), so can be more efficient to store and query with big tables.
DATE , DATETIME	SQL can also store date and time stamps to keep track of time series and event data. They can be tricky to work with especially when manipulating data across timezones.

Constraints

Constraint	Description
PRIMARY KEY	This means that the values in this column are unique, and each value can be used to identify a single row in this table.
AUTOINCREMENT	For integer values, this means that the value is automatically filled in and incremented with each row insertion. Not supported in all databases.
UNIQUE	This means that the values in this column have to be unique, so you can't insert another row with the same value in this column as another row in the table. Differs from the 'PRIMARY KEY' in that it doesn't have to be a key for a row in the table.
NOT NULL	This means that the inserted value can not be 'NULL'.
CHECK (expression)	This allows you to run a more complex expression to test whether the values inserted are valid. For example, you can check that values are positive, or greater than a specific size, or start with a certain prefix, etc.
FOREIGN KEY	This is a consistency check which ensures that each value in this column corresponds to another value in a column in another table. For example, if there are two tables, one listing all Employees by ID, and another listing their payroll information, the 'FOREIGN KEY' can ensure that every row in the payroll table corresponds to a valid employee in the master Employee list.

Practice-4

Create a new table named 'Database' with the following columns:

- Name: A string (text) describing the name of the database
- Version: A number (floating point) of the latest version of this database
- Download_count: An integer count of the number of times this database was downloaded

This table has no constraints.

Practice-4

Create a new table named 'Database' with the following columns:

- Name: A string (text) describing the name of the database
- Version: A number (floating point) of the latest version of this database
- Download_count: An integer count of the number of times this database was downloaded

This table has no constraints.

```
CREATE TABLE Database (  
    Name TEXT,  
    Version FLOAT,  
    Download_count INTEGER  
);
```

Tasks (in-class)

More practice in different sessions here

https://sqlbolt.com/lesson/select_queries_introduction

Solve:

- <https://leetcode.com/problems/second-highest-salary/>
- <https://leetcode.com/problems/employees-earning-more-than-their-managers/>
- <https://leetcode.com/problems/customers-who-never-order/>
- <https://leetcode.com/problems/department-highest-salary/>